

Database Systems and Laboratory

Logical Design

Nicola Saccomanno
`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

Database design

The process of designing a database is typically articulated into different phases:

1. Brainstorming meetings with IT personnel and all interested stakeholders
 - ▶ Collection of requirements
 - ▶ Design of a conceptual model (e.g., E-R model)
 - ▶ The E-R model has a specific notation, the *E-R diagram*, that helps all involved parts to discuss about the future database
2. Translation of the conceptual model into a logical model
 - ▶ Typically, a number of transformation is applied, followed by a set of translation rules
 - ▶ At this stage, a specific DBMS technology must be chosen (e.g., relational DB)
3. Addition to the logical model of details regarding the physical, low-level aspects (e.g., usage of indexes)
 - ▶ The physical schema is thus obtained

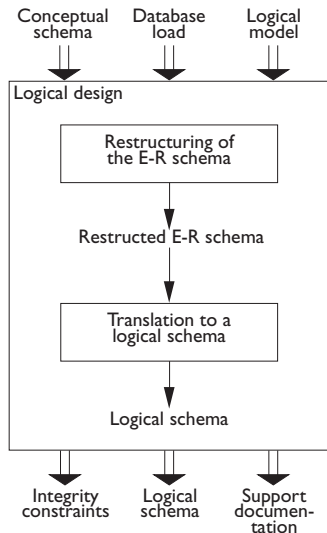
Some context

- ▶ We saw how to develop the conceptual schema of a database
- ▶ We have also seen the relational model as our logical model of choice
- ▶ In this set of slides, we will discuss the translation from the conceptual to the logical schema

Logical design steps

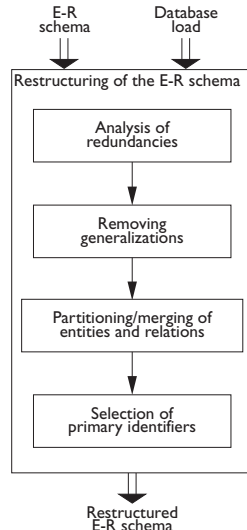
There are two main steps involved in the relational logical design process, when starting from a conceptual schema:

1. Entity-Relationship schema restructuring
2. Translation of the schema into a logical schema



Entity-Relationship schema restructuring

- ▶ It involves a simplification of the original E-R schema, to remove constructs that are not supported by the logical model
- ▶ The simplification operations consider also the intended use of the database, i.e., the operations that will be performed on the data
- ▶ The result is an E-R schema that takes into account some implementation aspects; thus, it is not, strictly speaking, a conceptual schema
- ▶ Very quick recap only, as you already saw the topic during the 1st semester

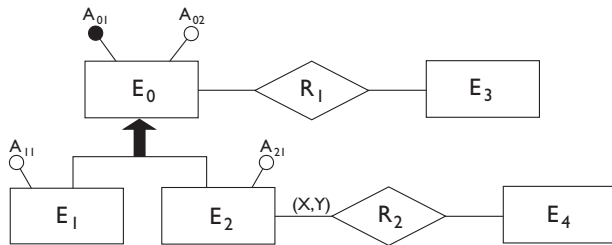


Analysis of redundancies

- ▶ A redundancy in a conceptual schema corresponds to a piece of information that can be derived (that is, obtained by a series of retrieval operations) from other data.
- ▶ An Entity-Relationship schema can contain various forms of redundancy.
- ▶ The decision to maintain or delete a redundancy is made by comparing the cost of operations that involve the redundant information and the storage needed, in the case of presence or absence of redundancy.

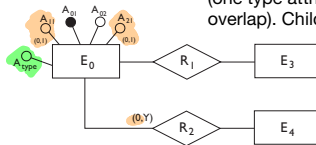
Removing generalizations / specializations

- ▶ The relational model does not allow the direct representation of generalizations / specializations of the E-R model.
- ▶ We need, therefore, to transform these constructs into other constructs that are easier to translate: entities and relationships.

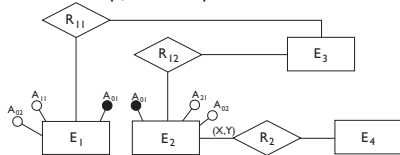


Three alternative for restructuring

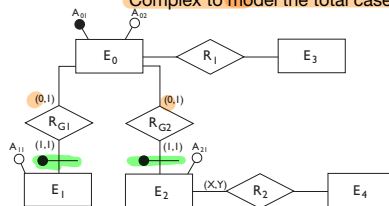
① Remove the children and put all info in the parent entity set, plus use a **type attribute** (one type attribute for each children, if with overlap). Children info become **optional**.



② Remove the parent and move all the information to the children. Possible if and only if the specialization is **total**. Note that, with overlap, we have possible inconsistencies



③ Hybrid solution. By means of weak entities, we can keep both parent and children. Resulting weak entity sets have **no partial key attributes**. Complex to model the total case



- ▶ We need to implement extra rule/constraints to ensure a coherent behaviour
- ▶ Mixing the three alternatives is also an option

Partitioning and merging of entities and relationships

- ▶ Entities and relationships of an E–R schema can be partitioned or merged to improve the efficiency of operations, using the following principle.
 - ▶ Accesses are reduced by separating attributes of the same concept that are accessed by different operations and by merging attributes of different concepts that are accessed by the same operations.
- ▶ The same criteria as those discussed for redundancies are valid in making a decision about this type of restructuring.
- ▶ Be careful when partitioning, as sometimes you could end up with a non-equivalent ER schema (you should introduce extra rules/constraint to ensure equivalence)

Removal of composite attributes

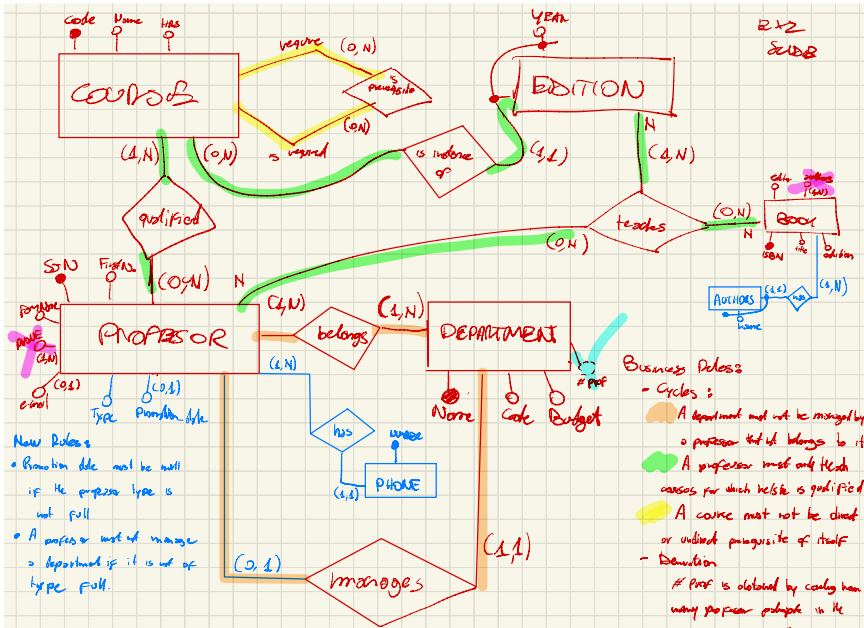
- ▶ Composite attributes **cannot be represented** in the relational model (first normal form property)
- ▶ Thus, we have two choices:
 - ▶ **Merge all composite** attributes together into a single one
 - ▶ **Keep only the composing** fields, modeling them as distinct attributes

Selection of primary identifiers

- ▶ The criteria for this decision, which follow the primary key discussion we had for the relation logical model, are:
 - ▶ Attributes with **null values cannot** form candidate keys.
 - ▶ **One or few attributes** are preferable to many attributes.
 - ▶ An internal identifier with few attributes is preferable to an external one, possibly involving many entities.
 - ▶ An identifier that is used by many operations to access the occurrences of an entity is preferable to others.
- ▶ At this stage, if none of the candidate identifiers satisfies the above requirements, it is possible to introduce a further attribute to the entity. This attribute will hold special values (often called codes) generated solely for the purpose of identifying occurrences of the entity.
 - ▶ Recall artificial / surrogate keys in the relational logical model
 - ▶ In practice, this is often done at the previous stage of the ER diagram design cycle

Exercise

- ▶ Let us now restructure the Entity-Relationship schema of Exercise 2, University DB
- ▶ Then, the Entity-Relationship schema of the Airport exercise!



Translation into a logical schema

- ▶ Now we have an E-R diagram
 - ▶ devoid of generalizations and multi-valued attributes, and
 - ▶ with a single identifier per entity set
- ▶ We can proceed with its translation into an *equivalent* relational logical schema
- ▶ The translation follows a set of pre-defined rules, that we will describe starting from the simplest case, i.e., that of entity sets

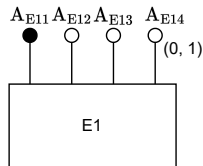
Translation into a logical schema

Entities

- ▶ Entities are just translated into single relations
- ▶ Recall that:
 - ▶ We already identified the (i.e., a single) primary key
 - ▶ We already removed composite, multi-valued, and ineffective derived attributes
- ▶ We need to add not-null and unique constraints whenever necessary

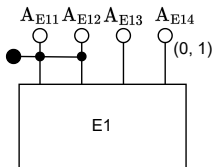
Entities translation into the logical schema

Notation



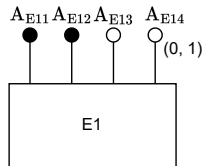
$E1(\underline{A_{E11}}, A_{E12}, A_{E13}, A_{E14})$

VNN : $\{A_{E12}, A_{E13}\}$



$E1(\underline{A_{E11}}, \underline{A_{E12}}, A_{E13}, A_{E14})$

VNN : $\{A_{E13}\}$

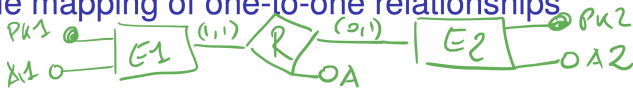


$E1(\underline{A_{E11}}, \underline{A_{E12}}, A_{E13}, A_{E14})$

VNN : $\{A_{E12}, A_{E13}\}$

UNIQUE : $\{A_{E12}\}$

The mapping of one-to-one relationships



We have to distinguish among 3 different cases:

- ▶ $E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$
 where $PK1$ is the key of entity $E1$, $A1$ is an attribute of $E1$,
 A is an attribute of relationship R , $PK2$ is the key of entity
 $E2$, and $A2$ is an attribute of $E2$
 (the case
 $E1(\mathbf{PK1}, A1) - (0, 1) - R(A) - (1, 1) - E2(\mathbf{PK2}, A2)$
 is completely symmetric, and thus ignored)
- ▶ $E1(\mathbf{PK1}, A1) - (0, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$
- ▶ $E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (1, 1) - E2(\mathbf{PK2}, A2)$

The case of $(1, 1) - (0, 1)$ relationships (AND VICE-VERSA)

How can we map the following ER schema into a corresponding relational one (introducing no redundancy, and preserving as much as possible information/constraints)?

$E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$



The case of $(1, 1) - (0, 1)$ relationships

How can we map the following ER schema into a corresponding relational one (introducing no redundancy, and preserving as much as possible information/constraints)?

$E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$

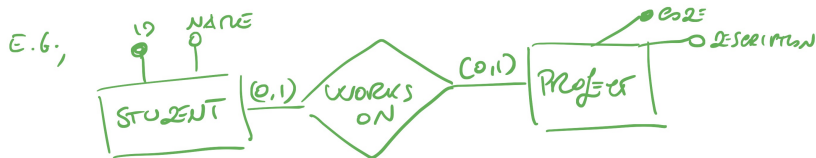
Relational schema:

$E1(\underline{PK1}, A1, PK2, A)$ and $E2(\underline{PK2}, A2)$, where $PK2$ is a foreign key of relation $E1$ that refers to the primary key $PK2$ of relation $E2$.

- ▶ $(1, _)$ on $E1$ side: $PK2$ NOT NULL in $E1$
- ▶ $(_, 1)$ on $E1$ side: $PK1$ is the primary key
- ▶ $(0, _)$ on $E2$ side: $PK2$ foreign key in $E1$ referring to the primary key of $E2$
- ▶ $(_, 1)$ on $E2$ side: $PK2$ UNIQUE in $E1$

The case of $(0, 1) - (0, 1)$ relationships

$E1(\mathbf{PK1}, A1) - (0, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$



The case of $(0, 1) - (0, 1)$ relationships

$E1(\mathbf{PK1}, A1) - (0, 1) - R(A) - (0, 1) - E2(\mathbf{PK2}, A2)$

Relational schema:

$E1(\underline{PK1}, A1, PK2, A)$ and $E2(\underline{PK2}, A2)$, where $PK2$ is a foreign key of relation $E1$ that refers to the primary key $PK2$ of relation $E2$.

- ▶ $(0, _)$ on $E1$ side: $PK2$ **can be NULL** in $E1$
- ▶ $(_, 1)$ on $E1$ side: $PK1$ is the primary key
- ▶ $(0, _)$ on $E2$ side: $PK2$ foreign key in $E1$ referring to the primary key of $E2$
- ▶ $(_, 1)$ on $E2$ side: $PK2$ UNIQUE in $E1$

$E1(\underline{PK1}, A1)$ and $E2(\underline{PK2}, A2, PK1, A)$, where $PK1$ is a foreign key of relation $E2$ that refers to the primary key $PK1$ of relation $E1$, works as well.

SO, BASICALLY IT IS THE SAME AS THE PREVIOUS SOLUTION, BUT WE DROP THE NOT-NULL CONSTRAINT

The case of $(0, 1) - (0, 1)$ relationships (contn'd)

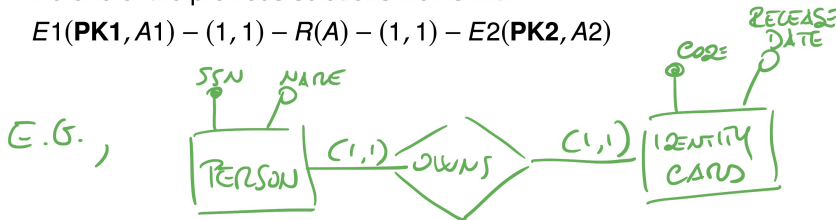
How do we choose between the two options? Participation of entities in the relationship can be a criterion.

→ EXTEND THE RELATION THAT CORRESPONDS TO THE ENTITY THAT "PARTICIPATES MOST" TO THE RELATIONSHIP, SO TO AVOID WASTING TOO MANY NULL VALUES.

The case of (1, 1) – (1, 1) relationships

No one of the previous solutions works with

$E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (1, 1) - E2(\mathbf{PK2}, A2)$



The case of $(1, 1) - (1, 1)$ relationships

No one of the previous solutions works with

$E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (1, 1) - E2(\mathbf{PK2}, A2)$

Relational schema:

$R(\underline{PK1}, A1, PK2, A2, A)$ (or $R(PK1, A1, \underline{PK2}, A2, A)$)

→ SIMPLY PUT ALL IN A SINGLE TABLE

→ EXCEPTION TO THE RULE "EACH ENTRY SET MAPS TO ITS OWN TABLE"

The case of $(1, 1) - (1, 1)$ relationships

No one of the previous solutions works with

$E1(\mathbf{PK1}, A1) - (1, 1) - R(A) - (1, 1) - E2(\mathbf{PK2}, A2)$

Relational schema:

$R(\underline{PK1}, A1, PK2, A2, A)$ (or $R(PK1, A1, \underline{PK2}, A2, A)$)

- ▶ $(1, _)$ on $E1$ side: $PK2$ NOT NULL in R
- ▶ $(_, 1)$ on $E1$ side: $PK1$ is the primary key (UNIQUE)
- ▶ $(1, _)$ on $E2$ side: $PK1$ is the primary key (NOT NULL)
- ▶ $(_, 1)$ on $E2$ side: $PK2$ UNIQUE in R

Translations from the E-R model to the relational (1)

Type	Initial schema	Possible translation
Binary many-to-many relationship		$E_1(\underline{A_{E11}}, A_{E12})$ $E_2(\underline{A_{E21}}, A_{E22})$ $R(\underline{A_{E11}}, \underline{A_{E21}}, A_R)$
Ternary many-to-many relationship		$E_1(\underline{A_{E11}}, A_{E12})$ $E_2(\underline{A_{E21}}, A_{E22})$ $E_3(\underline{A_{E31}}, A_{E32})$ $R(\underline{A_{E11}}, \underline{A_{E21}}, \underline{A_{E31}}, A_R)$
One-to-many relationship with mandatory participation		$E_1(\underline{A_{E11}}, A_{E12}, A_{E21}, A_R)$ $E_2(\underline{A_{E21}}, A_{E22})$

$FK: \{ (A_{E11}), (A_{E21}) \}$

$FK: \{ (A_{E11}), (A_{E21}), (A_{E31}) \}$

$FK: \{ (A_{E21}) \}$ $UNQ: \{ A_{E21} \}$

Translations from the E-R model to the relational (2)

Type	Initial schema	Possible translation
One-to-many relationship with optional participation		$E_1(\underline{A_{E11}}, A_{E12})$ $E_2(\underline{A_{E21}}, A_{E22})$ $R(\underline{A_{E11}}, A_{E21}, A_R)$ Alternatively: $E_1(\underline{A_{E11}}, A_{E21}, A_{E21}', A_R)$ $E_2(\underline{A_{E21}}, A_{E22})$
Relationship with external identifiers		$E_1(\underline{A_{E12}}, \underline{A_{E21}}, A_{E11}, A_R)$ $E_2(\underline{A_{E21}}, A_{E22})$

$FK: \{A_{E21}\}$

$FK: \{A_{E21}\}$ ~~$UNQ: \{A_{E21}\}$~~

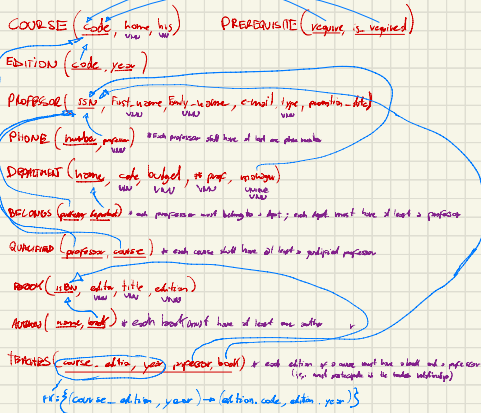
$FK: \{A_{E21}\}$

N.B. Additional constraints shall be added to reflect the proper semantic of the attributes

Exercise

- ▶ Let us now turn our restructured Entity-Relationship schema of Exercise 2, University DB, into a relational logical model
- ▶ You can try as exercise the translation of the restructured Entity-Relationship schema of the Airport exercise!

Translation University schema



- PK constraint
- PK constraint
- UNIQUE + NOT NULL + other constraint

References

Material for these slides is partly based on:

- ▶ Atzeni et al., *Database Systems: Concepts, Languages and Architectures*, Chapter 7 ENG / Chapter 8 ITA: "Logical Design" (book chapter and accompanying slides)